

Bases de Dados Relacionais e SQL II

Laboratórios de Sistemas e Serviços

Mário Antunes

Universidade de Aveiro

2026

Panorama das Bases de Dados SQL

Escolher a Base de Dados Correta I

Nem todos os RDBMS são criados da mesma forma. A escolha depende da escala do projeto.

- **Bases de Dados Embutidas (SQLite, H2):**

- Os dados são guardados num ficheiro local.
- Sem configuração de servidor.
- Perfeito para apps móveis, ferramentas CLI pequenas e testes.

- **Bases de Dados Baseadas em Servidor (MariaDB, PostgreSQL):**

- Corre como um processo separado (serviço).
- Lida com múltiplos utilizadores concorrentes de forma eficiente.
- Desempenho e funcionalidades de nível industrial.

Escolher a Base de Dados Correta II

- **MySQL / MariaDB:**
 - O mais popular para desenvolvimento web.
 - Rápido e fiável.
- **PostgreSQL:**
 - Conhecido pelas suas funcionalidades avançadas e integridade de dados.
- **Oracle / SQL Server:**
 - Bases de Dados comerciais para grandes empresas.
 - Extremamente caras mas com suporte de alto nível.

PostgreSQL: O “Rei” do Open Source

O PostgreSQL é frequentemente a escolha preferida dos engenheiros.

- **Tipos de Dados Avançados:** JSONB (para funcionalidades tipo NoSQL), Arrays, Geometria.
- **Extensibilidade:** Pode escrever as suas próprias funções em Python ou C.
- **Conformidade com o Padrão:** Segue o padrão SQL muito de perto.
- **Desempenho:** Altamente otimizado para consultas complexas e alta concorrência.

SQL Avançado: Junções e Conjuntos

Mergulho Profundo: Junções I

Na Aula 11 vimos Junções básicas. Vamos olhar para o panorama completo.

- **INNER JOIN:** Registos com valores correspondentes em ambos.
- **LEFT (OUTER) JOIN:** Tudo da esquerda + correspondências da direita.
- **RIGHT (OUTER) JOIN:** Tudo da direita + correspondências da esquerda.
- **FULL (OUTER) JOIN:** Tudo de ambos. (NULLs onde não há correspondência).

Mergulho Profundo: Junções II

- **CROSS JOIN:** Produto cartesiano (cada linha de A combinada com cada linha de B).
- **SELF JOIN:** Juntar uma tabela a si mesma (ex: tabela de funcionários onde manager_id referencia employee_id).

```
SELECT e.nome AS funcionario, m.nome AS gestor
FROM funcionarios e
LEFT JOIN funcionarios m ON e.manager_id = m.id;
```


Combinar os resultados de duas ou mais consultas.

- **UNION:** Combinar resultados e remover duplicados.
- **UNION ALL:** Combinar resultados e manter duplicados.
- **INTERSECT:** Apenas linhas presentes em ambos os resultados.
- **EXCEPT (ou MINUS):** Linhas no primeiro resultado mas não no segundo.

SQL Avançado: Agregações e Funções de Janela

Agregações: Resumir Dados

- **COUNT:** Número de linhas.
- **SUM:** Valor total.
- **AVG:** Valor médio.
- **MIN / MAX:** Menor e maior.

- **GROUP BY:** Agrupar linhas que têm os mesmos valores em linhas de resumo.
- **HAVING:** Filtrar grupos (como o WHERE mas para grupos).

```
SELECT dept_id, COUNT(*), AVG(salario)
FROM funcionarios
GROUP BY dept_id
HAVING AVG(salario) > 2000;
```

Common Table Expressions (CTEs)

As CTEs tornam as consultas complexas muito mais legíveis.

```
WITH vendas_regionais AS (  
    SELECT regiao, SUM(montante) AS total_vendas  
    FROM encomendas  
    GROUP BY regiao  
)  
SELECT regiao, total_vendas  
FROM vendas_regionais  
WHERE total_vendas > (SELECT AVG(total_vendas) FROM vendas_regionais);
```

- Atuam como “vistas temporárias” para uma única consulta.

Funções de Janela (Window Functions) I

As funções de janela realizam um cálculo num conjunto de linhas da tabela que estão de alguma forma relacionadas com a linha atual.

- Ao contrário do GROUP BY, elas **não** agrupam as linhas numa única linha de saída.

```
SELECT nome, salario,  
       AVG(salario) OVER(PARTITION BY dept_id) as salario_medio_dept  
FROM funcionarios;
```

- **RANK() / DENSE_RANK():** Classificar linhas dentro de uma partição.
- **ROW_NUMBER():** Atribuir um número único a cada linha.

Funções de Janela II: Ao longo do tempo

“Calcular o total acumulado de vendas.”

```
SELECT data, montante,  
       SUM(montante) OVER(ORDER BY data) as total_acumulado  
FROM vendas;
```

- Isto é extremamente poderoso para relatórios analíticos e dados de séries temporais.

Desempenho e Otimização

Índices I: A Necessidade de Velocidade

Como é que a base de dados encontra uma linha entre 10 milhões?

- Sem um índice: Percorre todas as linhas (**Sequential Scan**).
- Com um índice: Usa uma estrutura de dados (geralmente uma B-Tree) para a encontrar instantaneamente.

```
CREATE INDEX idx_nome_aluno ON alunos(nome);
```

- **Estratégia:** Indexar colunas usadas em WHERE, JOIN e ORDER BY.
- **EXPLAIN:** Use EXPLAIN ANALYZE <consulta> para ver como a base de dados executa o seu SQL e se está a usar índices.

Escala e Disponibilidade

O que acontece quando a sua base de dados é demasiado lenta ou demasiado grande para um único servidor?

- **Escalabilidade Vertical (Scale Up):** Comprar um servidor maior.
- **Escalabilidade Horizontal (Scale Out):** Adicionar mais servidores.

Copiar dados de um servidor (Primário) para outros (Réplicas).

- **Escala de Leitura:** O Primário lida com as escritas, as Réplicas com as leituras.
- **Failover:** Se o Primário falhar, uma Réplica torna-se o novo Primário.
- **Latência:** Síncrona (segura mas lenta) vs Assíncrona (rápida mas com risco de perda).

Sharding: Particionamento de Dados

Dividir uma tabela grande em pedaços menores (shards) através de diferentes servidores.

- **Estratégias:**
 - **Range Sharding:** IDs 1-1000 no S1, 1001-2000 no S2.
 - **Hash Sharding:** $id \% N$ para determinar o servidor.
- **Desafio:** Junções (Joins) entre shards são extremamente difíceis.

Segurança e Tendências Modernas

Injeção de SQL (SQL Injection)

NUNCA concatene entrada do utilizador no SQL.

MAU

```
query = "SELECT * FROM utilizadores WHERE nome = '" + user_input + "'";
```

BOM

```
cursor.execute("SELECT * FROM utilizadores WHERE nome = ?", (user_input,))
```

- Use **Prepared Statements** (consultas parametrizadas) para tratar a entrada como dados, não como código.

SQL vs NoSQL

Funcionalidade	SQL (Relacional)	NoSQL (Não-Relacional)
Esquema	Rígido / Predefinido	Flexível / Dinâmico
Relações	Junções (Complexas)	Desnormalizado (Aninhado)
Escala	Vertical (Principalmente)	Horizontal (Mais fácil)
Integridade	Conformidade ACID	BASE (Consistência Eventual)
Exemplos	PostgreSQL, MariaDB	MongoDB, Redis, Cassandra

Sumário

- **SQL Avançado:** Dominar Joins, CTEs e Funções de Janela.
- **Desempenho:** Usar Índices e EXPLAIN para otimizar consultas.
- **Escala:** Usar Replicação para leituras e Sharding para dados massivos.
- **Segurança:** Usar Prepared Statements para prevenir Injeção de SQL.
- **Escolha:** Usar SQL para dados estruturados e integridade; usar NoSQL para escala massiva ou esquemas flexíveis.